Reporte Final - Programación Concurrente

Facultad de Informática
UNIVERSIDAD NACIONAL DEL COMAHUE

- **Estudiantes:**
  - Contreras Ferrari, Matias Nahuel - FAI-5217
  - Ramos Montiel, Dante - FAI-5314

- **Materia:** Programación Concurrente
- **Profesora:** Silvia Amaro
- **Fecha:** 08/04/2026
- **Año de cursada**: 2025

```
├ 📂bin
│  ├ 📂activos
│  │  ├ 📂extras
│  │  │  ├ 📜AsistenteEspectaculo.class
│  │  │  ├ 📜AsistentePremios.class
│  │  │  ├ 📜Bicicleta.class
│  │  │  ├ 📜Camioneta.class
│  │  │  ├ 📜Gomones.class
│  │  │  ├ 📜Hora.class
│  │  │  └ 📜Tren.class
│  │  ├ 📂simulaciones
│  │  │  ├ 📜simulacionAutosC.class
│  │  │  └ 📜simulacionMontania.class
│  │  └ 📜Visitante.class
│  ├ 📂compartidos
│  │  ├ 📂atracciones
│  │  │  ├ 📜AutosChocadores.class
│  │  │  ├ 📜CarreraGomones.class
│  │  │  ├ 📜MontaniaRusa.class
│  │  │  └ 📜RealidadVirtual.class
│  │  ├ 📂extras
│  │  │  ├ 📜Mesa.class
│  │  │  └ 📜recorridoAGomones.class
│  │  ├ 📂shopping
```

```
│ │ │ ├ 📜AreaPremios.class
│ │ │ ├ 📜Comedor.class
│ │ │ ├ 📜Espectaculo.class
│ │ │ └ 📜Premio.class
│ │ └ 📜Parque.class
│ └ 📂main
│   └ 📜Main.class
├ 📂src
│ ├ 📂activos
│ │ ├ 📂extras
│ │ │ ├ 📜AsistenteEspectaculo.java
│ │ │ ├ 📜AsistentePremios.java
│ │ │ ├ 📜Bicicleta.java
│ │ │ ├ 📜Camioneta.java
│ │ │ ├ 📜Gomones.java
│ │ │ ├ 📜Hora.java
│ │ │ └ 📜Tren.java
│ │ ├ 📂simulaciones
│ │ │ ├ 📜simulacionAutosC.java
│ │ │ └ 📜simulacionMontania.java
│ │ └ 📜Visitante.java
│ ├ 📂compartidos
│ │ ├ 📂atracciones
│ │ │ ├ 📜AutosChocadores.java
```

```
│  │  │  ├📜CarreraGomones.java
│  │  │  ├📜MontaniaRusa.java
│  │  │  └📜RealidadVirtual.java
│  │  ├📂extras
│  │  │  ├📜Mesa.java
│  │  │  └📜recorridoAGomones.java
│  │  ├📂shopping
│  │  │  ├📜AreaPremios.java
│  │  │  ├📜Comedor.java
│  │  │  ├📜Espectaculo.java
│  │  │  └📜Premio.java
│  │  └📜Parque.java
│  └📂main
│     └📜Main.java
├📜Documentación tpfinal-concurrente (1).pdf
├📜README.md
└📜TPO  2025 -  Parque de Diversiones Grupal  ( 2 estudiantes).pdf
```

1. "El ingreso al parque está indicado a través del paso de k molinetes. Una vez ingresado, el visitante puede optar por ir al shopping o disfrutar de las actividades del parque"

En nuestra implementación, el constructor del recurso compartido Parque contiene un arreglo de Semaphore[] llamado `molinetes`, donde cada semáforo representa un molinete individual con 1 permiso inicial. Además, se utiliza un semáforo `chequeoHorario` para proteger el acceso concurrente a las variables de estado del parque (`abierto` y `expulsarVisitantes`).

El método `tomarMolinete()` primero adquiere el semáforo `chequeoHorario` para verificar si el parque está abierto y si no se está expulsando visitantes. Si las condiciones son favorables, recorre el arreglo de molinetes utilizando `tryAcquire()` hasta encontrar uno disponible. En caso de que ningún molinete esté libre después de recorrer todo el arreglo, el hilo se bloqueará en un molinete elegido aleatoriamente mediante `acquire()` hasta que se libere.

El método retorna un entero que representa el índice del molinete tomado, o -1 si el parque está cerrado. Posteriormente, el visitante llama a `dejarMolinete(int i)` para liberar el semáforo correspondiente mediante `release()`.

---

2. "El complejo se encuentra abierto para el ingreso de 09:00 a 18:00hs. Considere que las actividades cierran a las 19.00 hrs. y a las 23hs no debería quedar nadie en el parque."

En nuestra implementación, la clase Parque contiene tres variables booleanas para controlar el estado del parque: `abierto` (indica si se permiten nuevos ingresos por molinete), `atraccionesAbiertas` (controla si las atracciones están operativas) y `expulsarVisitantes` (señal para evacuar el parque a las 23hs). Además, mantiene una variable `horaActual` que representa la hora simulada.

El hilo `Hora` se encarga de modificar periódicamente la hora del parque mediante el método `cambiarHora()`, el cual incrementa `horaActual` en una unidad cada 5 segundos. Dentro de este método, un switch evalúa la hora actual y ejecuta las acciones correspondientes: a las 9:00 se abre el parque y las atracciones mediante `abrirAtracciones()`, a las 18:00 se

cierra el ingreso (pero las atracciones siguen funcionando), a las 19:00 se cierran las atracciones mediante `cerrarAtracciones()`, y a las 23:00 se activa `expulsarVisitantes` y se llama a `hecharGente()` para evacuar el parque.

Los visitantes pueden esperar la reapertura del parque usando el método `esperarApertura()`, que bloquea al hilo en un `aperturaLock` mediante `wait()` hasta que el parque vuelva a abrir o cierre definitivamente.

---

3. "La montaña rusa tiene una capacidad máxima de 5 personas y el viaje solo puede iniciar cuando todos los asientos están ocupados. Los visitantes deben esperar a que el viaje anterior haya finalizado antes de poder subir a la atracción. Para este juego existe un espacio de espera destinado a los visitantes que desean subir a la montaña rusa. Si dicho espacio de espera se encuentra completo, el visitante abandona la fila y se dirige a otra atracción. Como recompensa por utilizar esta atracción, el visitante recibe MR fichas."

En nuestra implementación, se utiliza una `BlockingQueue` con capacidad de 10 elementos para simular la cola de espera. El control de la atracción se realiza mediante cuatro semáforos:

• `habilitado`: semáforo con 5 permisos iniciales que controla el ingreso de visitantes a cada tanda.
• `todosSentados`: barrera de sincronización que bloquea al hilo simulador hasta que los 5 asientos estén ocupados.
• `esperarViaje`: mantiene bloqueados a los visitantes sentados mientras dura el viaje.
• `todosBajaron`: coordina que todos los visitantes hayan descendido antes de iniciar la siguiente tanda.

El método `entrar(Visitante v)` intenta agregar al visitante a la cola mediante `offer()`, retornando false si está llena o si la atracción está cerrada. El método `subirAlVagon`(`int id`) primero adquiere un permiso de `habilitado`, verifica si la atracción sigue abierta, incrementa el contador `sentados` (usando AtomicInteger), libera un permiso en `todosSentados` y luego espera en `esperarViaje` hasta que termine el recorrido. Finalmente llama a `bajar()` que libera un permiso en `todosBajaron`.

El hilo simulador ejecuta `iniciarViaje()` que adquiere 5 permisos de `todosSentados`, y `terminarViaje()` que libera 5 permisos en `esperarViaje`, espera 5 permisos de `todosBajaron`, y finalmente libera 5 permisos en `habilitado` para habilitar la siguiente tanda. Al cerrar la atracción, se limpian los semáforos y la cola para evitar bloqueos.

---

4. "Los autitos chocadores requieren dos personas por auto y la atracción dispone de un máximo de 10 autos; el juego no puede comenzar hasta que todos los autos estén ocupados, lo que implica que cada turno requiere un total de 20 personas, por lo que los visitantes deben esperar hasta que se complete la ocupación de todos los autos para que la atracción pueda iniciar; como recompensa por participar en este juego, el visitante recibe AC fichas."

En nuestra implementación, esta atracción se gestiona mediante un monitor utilizando synchronized junto con wait() y notifyAll(). Se utilizan tres contadores: `esperando` (personas sentadas), `bajaron` (personas que descendieron), y `asientos` (para asignar autos). Además, hay una variable booleana `enCurso` que indica si la atracción está en funcionamiento.

El método `entrarAutosChocadores`(`int id`) primero verifica si la atracción está abierta. Si hay 20 personas esperando y la atracción está abierta, el hilo espera mediante `wait`(). Cuando hay espacio, el visitante se sienta en un auto (calculado como asientos / 2), incrementa los contadores, y si es el visitante número 20, despierta al encargado con notifyAll(). Luego espera a que inicie la atracción (`enCurso` = `true`), espera a que termine, y finalmente llama a bajar() que incrementa bajaron y despierta al encargado cuando los 20 han descendido.

El método `iniciarAutos`() espera hasta que esperando sea 20, establece `enCurso` = `true` y despierta a los pasajeros. El método `terminarAutos`() establece `enCurso` = `false`, espera a que bajaron sea 20, reinicia los contadores y habilita la siguiente tanda mediante `notifyAll`().

---

5. "La actividad de realidad virtual del parque requiere que cada visitante cuente con un equipo completo compuesto por un visor de realidad virtual, dos manoplas y una base, los cuales deben ser proporcionados

por el encargado de la atracción antes de que el visitante pueda participar. La cantidad disponible de cada uno de estos elementos es limitada y puede variar, por lo que solo se permite el ingreso de visitantes que reciban un equipo completo; si falta alguno de los componentes necesarios, el visitante debe esperar hasta que se encuentre disponible. Al finalizar su participación en la actividad, el visitante recibe RV fichas."

En nuestra implementación se utiliza un `ReentrantLock` junto con una `Condition` llamada `equipoDisponible` para controlar el acceso a los recursos. Se mantienen tres contadores: `visores`, `manoplas` y `bases`, que representan la cantidad disponible de cada elemento.

El método `entrar(int id)` adquiere el lock y verifica si hay recursos suficientes para formar un equipo completo (1 visor, 2 manoplas, 1 base). Si algún recurso es insuficiente y la atracción está abierta, el hilo espera mediante `equipoDisponible.await()`. Cuando hay recursos disponibles, se decrementan los contadores correspondientes y se libera el lock. Si la atracción se cierra mientras el visitante espera, se lanza una `InterruptedException`.

El método `salir(int id)` adquiere el lock, incrementa los contadores de recursos devueltos (visores, manoplas y bases), despierta a los hilos en espera mediante `equipoDisponible.signalAll()`, libera el lock y retorna 19 puntos. De esta manera se garantiza que ningún visitante pueda iniciar la actividad sin contar con todos los componentes requeridos.

---

6. "Carrera de gomones por un río artificial…"

En nuestra implementación, la actividad de carrera de gomones utiliza una combinación de semáforos, colas bloqueantes y estructuras concurrentes para coordinar tres elementos principales: el transporte de pertenencias, la disponibilidad de gomones, y la sincronización de las carreras.

Cada visitante que participa entrega un bolso que se almacena en un `ConcurrentHashMap`<Integer, Object> llamado Bolsos, donde la clave es el ID del visitante. El transporte de estos bolsos se coordina mediante dos semáforos: `Camion` (que señala cuándo la camioneta puede partir) y

`EsperarCamion` (que bloquea a los visitantes hasta que el transporte llegue al destino).

Los gomones disponibles se gestionan mediante una `BlockingQueue` llamada `colaGomones`. Cada gomón tiene asociado un semáforo individual almacenado en un `ConcurrentHashMap`<Integer, Semaphore> llamado `semGomon`, que se utiliza para coordinar la liberación del visitante cuando termina su carrera.

El método `cicloGomones`(int `gomonId`, int `cantVisitantes`) implementa un ciclo de sincronización en tres fases mediante barreras construidas con semáforos (`Largada`, `LargadaCarrera`, `ReunionFinal`): en la Fase 1, los gomones llegan a la estación y esperan hasta que estén todos (`cantGomonesNecesarios`); en la Fase 2, cada gomón toma sus pasajeros de `colaGomones`; en la Fase 3, cuando todos están cargados, el último gomón en estar listo libera el semáforo `Camion` y `LargadaCarrera` para que arranquen simultáneamente la camioneta y la carrera.

El método `finCicloGomones`(int gomonId, int[] visitantes) determina atómicamente el ganador usando `AtomicBoolean.compareAndSet`(), libera a sus visitantes, y coordina la reunión final mediante la barrera `ReunionFinal`. El método `esperarBolsosCamion`() bloquea al hilo camioneta hasta recibir la señal de `Camion`, y `finViajeCamion`() libera exactamente `bolsosEnTransito` permisos en `EsperarCamion` para que los visitantes recuperen sus bolsos.

---

7. "Área de Premios…"

En nuestra implementación, el área de premios utiliza dos objetos `Exchanger` para realizar un intercambio de información en dos fases entre el visitante y el asistente. El primer exchanger (`exchangerPuntos`) se usa para que el visitante entregue sus puntos, y el segundo (`exchangerSaldo`) para que el asistente devuelva el saldo restante.

El método `canjearPremio`(int `id`, int `puntosEntregados`) es llamado por el visitante. Primero verifica si el área está abierta; si está cerrada, retorna los puntos sin modificar. En la Fase 1, envía los puntos al asistente mediante `exchangerPuntos.exchange`() con un timeout de 3 segundos. En la Fase 2, espera recibir el saldo restante mediante `exchangerSaldo.exchange`(),

también con timeout. Si ocurre un `TimeoutException` (porque el asistente ya no está activo), el visitante conserva sus puntos.

El método `atenderVisitante`(`IntUnaryOperator` calcularSaldo) es ejecutado por el hilo asistente. En la Fase 1, recibe los puntos del visitante mediante `exchangerPuntos`.`exchange`(). Luego aplica la función `calcularSaldo` para determinar el saldo a devolver según la lógica de premios. En la Fase 2, devuelve el saldo mediante `exchangerSaldo`.`exchange`(). Si no hay visitante esperando, el timeout permite que el asistente continúe sin bloquear indefinidamente.

El uso de `Exchanger` permite modelar de forma natural la interacción sincronizadaa entre visitante y encargado, ya que ambos deben encontrarse simultáneamente para realizar el intercambio. Al cerrar el área, se recrean nuevos exchangers para evitar que hilos queden bloqueados esperando un intercambio que no ocurrirá.

---

8. "Comedor del parque…"

En nuestra implementación, el comedor está compuesto por un arreglo de objetos Mesa, cada uno con capacidad para 4 personas. Cada mesa gestiona su propia sincronización de forma independiente. El objeto Comedor utiliza un semáforo `mutex` para proteger el acceso concurrente durante la asignación de mesas.

El método `entrar`() adquiere el `mutex` y recorre el arreglo de mesas buscando una que no esté ocupada mediante `Mesa`.`estaOcupada`(). Cuando encuentra una disponible, libera el `mutex` temporalmente y llama al método `Mesa`.`usar`(), que devuelve `true` si pudo sentarse o `false` si la mesa se llenó antes. Si no pudo sentarse, vuelve a adquirir el `mutex` para continuar buscando. Si todas las mesas están ocupadas, retorna un resultado indicando que no pudo ingresar.

La clase Mesa utiliza un semáforo mutex para proteger sus contadores y una `CyclicBarrier` con capacidad de 4 para sincronizar a los comensales. El método `Mesa`.`usar`() incrementa el contador actual, y cuando llega a 4, marca la mesa como ocupada (`ocupada` = `true`). Luego espera en la barrera mediante `await`() (fuera del `mutex` para evitar deadlock) hasta que los 4

visitantes estén sentados. La barrera tiene una acción de finalización que imprime un mensaje indicando que el grupo completo comienza a comer.

El método `Mesa.dejar()` decrementa el contador actual, y cuando llega a 0, marca la mesa como disponible (`ocupada` = `false`) para que un nuevo grupo pueda usarla. De esta manera se garantiza que todos los visitantes de una mesa comiencen a comer al mismo tiempo y que la mesa se libere completamente antes de ser reutilizada.

---

9. "Espectáculo en el Teatro…"

En nuestra implementación, el espectáculo del teatro se controla mediante un `ReentrantLock` y tres objetos `Condition` que permiten coordinar la entrada de visitantes en grupos de 5, la sincronización de asistentes, y la ejecución del espectáculo. El teatro tiene una capacidad máxima de 20 personas.

El método `iniciarEspectaculo()` establece `hayEspectaculo` = `true`, `puertasCerradas` = `false`, y reinicia los contadores `visitantesEntrados` y `esperandoGrupo` a 0. Este método es llamado por el hilo `Hora` en los horarios correspondientes (9:00, 12:00, 15:00, 18:00, 21:00).

El método `entrarVisitante`(int `idVisitante`) primero verifica las condiciones de ingreso. Si el espectáculo no está disponible o las puertas están cerradas, el visitante no puede entrar. Si puede intentar, incrementa `esperandoGrupo` y espera en la condición `grupoCompletoCond` hasta que se forme un grupo de 5. Cuando `esperandoGrupo` llega a 5, el quinto visitante incrementa `visitantesEntrados` en 5, resetea `esperandoGrupo` a 0, y despierta a todos mediante `grupoCompletoCond.signalAll()`. Si se alcanza la capacidad máxima de 20, se cierran las puertas automáticamente.

El método `empezarEspectaculo()` es llamado 1 hora después de abrir las puertas. Establece `puertasCerradas` = true, despierta a visitantes que no completaron su grupo mediante `grupoCompletoCond.signalAll()`, y avisa a los asistentes que pueden entrar mediante `asistentesListos.signalAll()`.

El método `asistenteEsperaVisitantes`() bloquea al hilo asistente en la condición `asistentesListos` hasta que las puertas se cierren. Una vez que las puertas están cerradas, el método `entrarAsistentes`() establece `espectaculoEnCurso` = `true`. Los visitantes y asistentes esperan en `showTerminadoCond` hasta que el método `finEspectaculo`() establezca `hayEspectaculo` = `false` y `espectaculoEnCurso` = `false`, liberando a todos mediante `showTerminadoCond`.`signalAll`().

Este mecanismo garantiza que: (1) los visitantes entren siempre en grupos completos de 5, (2) nunca se supere la capacidad máxima de 20 personas, (3) los asistentes entren solo cuando las puertas estén cerradas, y (4) todos esperen hasta que el espectáculo haya finalizado antes de retirarse.

---

10. Recorrido a Gomones (Tren y Bicicletas)

En nuestra implementación, el recorrido hacia la zona de gomones ofrece dos medios de transporte: un tren colectivo y bicicletas individuales. Ambos se gestionan mediante la clase `recorridoAGomones`.

Tren: El tren funciona como transporte colectivo para 15 visitantes. Se utilizan dos semáforos: `visitantesEnTren` (contador de pasajeros) y `trenListo` (señal de finalización del viaje). El método `subirTren`(int `id`) incrementa el contador mediante `visitantesEnTren`.`release`() y luego espera en `trenListo`.`acquire`() hasta que el viaje termine. El hilo tren ejecuta `cicloTren`(), que espera a reunir 15 permisos en `visitantesEnTren` mediante `acquire`(15), simula el trayecto con un `sleep` de 3 segundos, y luego libera 15 permisos en `trenListo` mediante `release`(15) para despertar a todos los pasajeros simultáneamente.

Bicicletas: Las bicicletas funcionan de forma individual y bajo demanda. Se utiliza una `BlockingQueue` llamada `colaBici` para encolar las solicitudes de los visitantes, y un `HashMap`<`Integer`, `Semaphore`> llamado `semBici` para asociar a cada visitante con su semáforo de sincronización. El método `usarBicicleta`(int `id`) crea un semáforo, lo registra en el mapa, encola el ID del visitante en `colaBici`, y espera en el semáforo hasta ser liberado. Cada hilo bicicleta ejecuta `cicloBicicleta`(int `bikeId`), que extrae un visitante de la cola con un timeout de 500ms, simula el paseo con un `sleep`

de 2 segundos, y luego libera el semáforo del visitante mediante `release`(), permitiéndole continuar.

Al cerrar el recorrido mediante `cerrar`(), se liberan ambos semáforos del tren y todos los semáforos de bicicletas para evitar que hilos queden bloqueados. El método `abrir`() reactiva el servicio.